

IFT436 - Devoir #1

À remettre avant le **vendredi 25 septembre avant 23h59**.

Tout retard entraînera jusqu'à 33% de pénalité par jour.

Pondération. Ce travail compte pour 6% des points de la session.

Modalités. Considérez les points suivants.

- Le travail peut être fait seul ou en équipe de deux.
- Le devoir doit être remis en format pdf via turnin. La provenance de votre pdf n'a pas d'importance — ce pdf peut provenir d'un export d'un fichier Word ou d'un fichier texte, d'un scan de vos écrits/dessins, etc.

Il n'y a pas de norme de présentation particulière. Écrivez lisiblement, et assurez-vous seulement que **vos noms et CIP apparaissent clairement sur le devoir remis**.

- Toute argumentation qui démontre l'énoncé correctement sera acceptée, que ce soit sous la forme de série formules mathématiques, ou bien d'explications textuelles (ou un mélange des deux).
- Vous pouvez utiliser les techniques de preuves que vous désirez, vous n'êtes pas dans l'obligation de suivre les suggestions (mais je le recommande fortement).
- Vous pouvez utiliser les algorithmes et résultats démontrés en classe et dans les solutionnaires d'exercices sans avoir à les démontrer. Par exemple, si vous voulez utiliser le tri par insertion, vous pouvez l'invoquer sans écrire le pseudo-code de l'algorithme. Par contre, si vous utilisez de nouveaux résultats, vous devez les démontrer.

Question 1 : une question d'échauffement (15 points)

Dans le problème de la *sélection*, on veut trouver la k -ième plus petite valeur dans un tableau donné contenant des valeurs distinctes (le tableau n'est pas nécessairement trié).

Problème de la sélection

Entrée: un tableau T d'entiers contenant n éléments distincts, et un entier $k \in \{1, 2, \dots, n\}$.

Sortie: un indice $p \in \{1, 2, \dots, n\}$ tel que

$$|\{q : 1 \leq q \leq n \wedge T[q] < T[p]\}| = k - 1.$$

Par exemple, si $T = [7, 9, 0, 5, 10, 2, 11]$ et $k = 3$, il faut retourner 4 parce que $T[4] = 5$ est la troisième plus petite valeur (il y a 0 et 2 qui sont plus petits que 5). Autre exemple, si $k = 5$ alors il faut retourner 2 car $T[2] = 9$.

Donnez le pseudo-code d'un algorithme qui résout le problème de la sélection. Donnez ensuite la complexité en temps de votre algorithme avec la notation O , sans justification.

Vous aurez un maximum de points pour un algorithme en temps $O(n \log n)$. Vous n'avez pas à justifier que votre algorithme est correct.

Solution.

Il suffit de trier T en temps $O(n \log n)$ et de retrouver en temps linéaire la position du T original qui contient la k -ième valeur. Il était aussi possible de former des couples de la forme $(valeur, position)$ et de trier selon les valeurs, puis de retourner la position du k -ième couple.

```
fonction selection( $T, k$ )
|    $T' = tri\acute{f}usion(T);$ 
|   pour  $i = 1$  à  $|T|$  faire
|       |   si  $T[i] == T'[k]$  alors
|           |   return  $i$ ;
|       |   fin
|   fin
|   return  $null$ ;
```

Question 2 : le problème de la sous-somme (50 points)

Dans le problème de la sous-somme, on a un ensemble d'entiers et on veut savoir si on peut trouver un sous-ensemble d'éléments qui somment à t .

Problème de la sous-somme

Entrée: un ensemble S d'entiers positifs et un entier $t > 0$.

Sortie: un sous-ensemble $S' \subseteq S$ tel que $\sum_{s \in S'} s = t$, ou bien *null* s'il n'existe pas de tel S' .

a. Considérez l'algorithme suivant.

```
fonction algo1( $S, t$ )
    si  $\sum_{s \in S} s < t$  alors return null;
     $q = 0$ ;
     $S' = \emptyset$ ;
    tant que  $q < t$  faire
        Soit  $m$  l'élément maximum de  $S$ ;
         $q = q + m$ ;
        Retirer  $m$  de  $S$ ;
        Ajouter  $m$  à  $S'$ ;
    fin
    si  $q == t$  alors
        return  $S'$ ;
    sinon
        return null;
    fin
```

Est-ce que cet algorithme résout correctement le problème de la sous-somme? Démontrez votre réponse.

Solution.

Non, cet algorithme ne résout pas correctement le problème de la sous-somme. Nous allons le montrer en fournissant le contre-exemple suivant: Soient $S = \{1, 3, 5\}$ et $t = 4$. Remarquez qu'il existe un sous-ensemble $\{1, 3\}$ de S dont la somme des éléments est égale à $t = 4$. Par contre, si nous exécutons notre algorithme sur cette entrée d'abord $q = 0$ et $S' = \emptyset$. Ainsi, $q < t = 4$ alors $m = 5$ sera l'élément maximum, $q = 0 + 5 = 5$, $S = \{1, 3\}$ et $S' = \{5\}$. Ensuite, comme $q \neq t$, l'algorithme retournera nil. Mais ce résultat n'est pas correct puisque nous savons qu'il existe le sous-ensemble $\{1, 3\}$ qui satisfait la condition.

b. Considérez l'algorithme suivant.

```
fonction algo2( $S, t$ )
    pour chaque sous-ensemble  $S'$  de  $S$  faire
        si les éléments de  $S'$  somment à  $t$  alors
            return  $S'$ ;
        fin
    fin
    return null;
```

Cette question comporte deux sous-questions:

- (1) Est-ce que cet algorithme résout correctement le problème de la sous-somme? Démontrez votre réponse.
- (2) Combien de fois est-ce que l'algorithme entrera dans la boucle **pour chaque**, dans le pire cas en fonction de $n = |S|$? Justifiez votre réponse.

Solution.

Oui cet algorithme résout correctement le problème de la sous-somme car il analyse chaque sous-ensemble qu'on peut former avec les éléments de S . Donc, s'il existe un sous-ensemble qui atteint la somme cible, il sera forcément trouvé. De plus, si un tel sous-ensemble n'existe pas, aucun tour de la boucle ne retournera d'ensemble et l'algorithme retournera correctement *null*.

La condition sera évaluée dans le pire cas 2^n fois où n est la taille de S , parce que c'est le nombre de sous-ensembles possible. Ce pire se présente lorsqu'il n'existe pas de sous-ensemble de somme t .

- c. Considérez maintenant la variante suivante du problème de la sous-somme, dans laquelle on nous spécifie la taille du sous-ensemble désirée.

Problème de la sous-somme de taille désirée

Entrée: un ensemble S d'entiers positifs, un entier $k \in \{0, 1, \dots, |S|\}$ et un entier $t > 0$.

Sortie: un sous-ensemble $S' \subseteq S$ tel que $|S'| = k$ et tel que $\sum_{s \in S'} s = t$, ou bien *null* s'il n'existe pas de tel S' .

L'algorithme ci-haut peut être modifié pour énumérer seulement les sous-ensembles de la taille voulue.

```
fonction algo3( $S, t, k$ )  
    pour chaque sous-ensemble  $S'$  de  $S$  tel que  $|S'| = k$  faire  
        si les éléments de  $S'$  somment à  $t$  alors  
            return  $S'$ ;  
        fin  
    fin  
    return null;
```

Cette question comporte deux sous-questions:

(1) Combien de fois est-ce que l'algorithme entrera dans la boucle **pour chaque**, dans le pire cas en fonction de k et $n = |S|$? Justifiez votre réponse.

(2) Est-ce que la valeur de votre réponse en c.(1) est plus petite que la valeur de votre réponse en b.(2), pour $n > 0$? Justifiez votre réponse.

Solution.

On entrera dans la boucle au plus $\binom{n}{k}$ fois, qui est par définition le nombre de sous-ensembles de taille k d'un ensemble de taille n .

Pour $n > 0$, on a que $\binom{n}{k} < 2^n$, parce que tel que vu en exercice,

$$2^n = \sum_{i=0}^n \binom{n}{i}.$$

Donc, $\binom{n}{k}$ n'est qu'un seul des termes de cette sommation, et puisque $n > 0$, il y a plus qu'un terme non-zéro dans cette somme, d'où $\binom{n}{k} < 2^n$.

Question 3: puissances en temps logarithmique (35 points)

Soient x et k deux entiers tels que $x \geq 1$ et $k \geq 0$. La fonction $\text{pow}(x, k)$ retourne la valeur de x^k . Elle est implémentée par l'algorithme suivant:

```
fonction pow(x, k)
|   si k == 0 alors
|       Retourner 1;
|   sinon si k == 1 alors
|       Retourner x;
|   sinon
|       tmp = pow(x, [k/2]) ;           // [k/2] = valeur plancher de k/2
|       si k%2 == 0 alors
|           Retourner tmp * tmp;
|       sinon
|           Retourner tmp * tmp * x;
|   fin
fin
```

Rappelons que $\%$ est le *modulo*, c'est-à-dire le reste de la division entière. On a $k\%2 == 0$ si et seulement si k est un nombre pair.

Cet algorithme est intéressant, car le nombre d'appels total est logarithmique en k .

- a. Démontrez que cet algorithme calcule correctement x^k .

Suggestion: par induction sur k , commencez avec $k = 0$ et $k = 1$ comme cas de base.

Solution.

Dans cette démonstration, on va dénoter par $\text{pow}(x, k)$ la valeur retournée par l'algorithme sur entrée x et k . Nous allons démontrer que $\text{pow}(x, k) = x^k$ pour tout x, k tels que $x \geq 1$ et $k \geq 0$. On fera cette démonstration par induction sur k .

Cas de base: On commence pour prouver pour $k = 0$. On a $\text{pow}(x, 0) = 1$ car l'algorithme retourne 1 quand $k = 0$ (première ligne), et aussi $x^0 = 1$. Ensuite, pour $k = 1$ on a $\text{pow}(x, 1) = x$ et aussi $x^1 = x$. Ce qui démontre l'égalité dans les deux cas.

Induction: Nous énonçons notre hypothèse d'induction en disant: On suppose que pour tout entier x et j tels que $x \geq 1$ et $0 \leq j < k$ avec $j, k \in \mathbb{N}$, $\text{pow}(x, j) = x^j$. Puis on doit montrer que cela implique que $\text{pow}(x, k) = x^k$. Il y a deux cas:

- Quand $k\%2 == 0$, autrement dit, quand k est *pair*. Dans ce cas, $[k/2] = k/2$ et l'algorithme appelle $\text{pow}(x, k/2)$ et le multiplie avec lui-même. Nous avons donc:

$$\begin{aligned} \text{pow}(x, k) &= \text{pow}(x, [k/2]) \cdot \text{pow}(x, [k/2]) \\ &= \text{pow}(x, k/2) \cdot \text{pow}(x, k/2) \\ &= x^{k/2} \cdot x^{k/2} \quad (\text{parce que } k/2 < k, \text{ et par hypothèse d'induction}) \\ &= x^k \end{aligned}$$

- Quand $k\%2 \neq 0$, autrement dit, quand k est *impair*. Dans ce cas, $[k/2] = (k-1)/2$ et l'algorithme appelle $\text{pow}(x, (k-1)/2)$, puis multiplie cette valeur avec elle-même

puis avec x . Nous avons donc:

$$\begin{aligned}
 \text{pow}(x, k) &= \text{pow}(x, \lfloor k/2 \rfloor) \cdot \text{pow}(x, \lfloor k/2 \rfloor) \cdot x \\
 &= \text{pow}(x, \frac{k-1}{2}) \cdot \text{pow}(x, \frac{k-1}{2}) \cdot x \\
 &= x^{\frac{k-1}{2}} \cdot x^{\frac{k-1}{2}} \cdot x \quad (\text{parce que } \frac{k-1}{2} < k, \text{ et par hypothèse d'induction}) \\
 &= x^{\frac{2(k-1)}{2}} \cdot x \\
 &= x^{k-1} \cdot x = x^k
 \end{aligned}$$

Donc, cet algorithme calcule correctement x^k .

1

- b. Soit $r(k)$ le nombre d'appels à la fonction effectués lors de l'exécution, en comptant l'appel initial. Par exemple, $r(1) = 1$ et $r(4) = 3$. Nous allons supposer que k est une puissance de 2.

Montrez de façon rigoureuse que pour tout $k \in \mathbb{N}$ tel que k est une puissance de 2, on a $r(k) = \log(k) + 1$.

Suggestion: Représentez $r(k)$ par une récurrence et résolvez cette dernière par induction sur les exposants des puissances de 2, avec $k = 2^0 = 1$ comme cas de base.

Solution.

En supposant que k est une puissance de 2, on a $k \geq 1$. Nous allons d'abord représenter $r(k)$ par une récurrence:

$$r(k) = \begin{cases} 1 & \text{si } k = 1, \\ r(k/2) + 1 & \text{sinon.} \end{cases}$$

Pour justifier cette récurrence, on voit que si $k = 1$, l'algorithme retourne immédiatement et un seul appel est fait (donc $r(1) = 1$). Quand $k \geq 2$, on doit compter l'appel initial, plus les $r(k/2)$ appels fait par la récursion avec $k/2$, qui est un entier car k est une puissance de 2. Notez que le domaine de r est bel et bien l'ensemble des puissances de 2, car $k/2$ est aussi une puissance de 2.

Nous allons montrer que pour tout $k \in \mathbb{N}$ pour lequel il existe $j \geq 0$ tel que $k = 2^j$, $r(k) = \log(k) + 1$. On fera cette démonstration par induction sur l'exposant j .

Cas de base: Lorsque $j = 0$ on a $k = 2^0 = 1$. Alors $r(k) = r(2^0) = r(1) = 1$. De l'autre côté, on aura $\log(k) + 1 = \log(1) + 1 = 0 + 1 = 1$. L'égalité est donc vérifiée.

Induction: On suppose que pour tout $l < k$ tel que l est une puissance de 2, on a $r(l) = \log(l) + 1$. On veut montrer que si $k = 2^j$, alors $r(k) = \log(k) + 1$. En utilisant notre

définition de $r(k)$, on a

$$\begin{aligned}r(k) &= r(k/2) + 1 \\&= r\left(\frac{2^j}{2}\right) + 1 \\&= r(2^{j-1}) + 1 \\&= (\log(2^{j-1}) + 1) + 1 \quad \text{parce que } 2^{j-1} < 2^j, \text{ et par hypothèse d'induction.} \\&= \log(k/2) + 2 \\&= \log(k) - \log(2) + 2 \\&= \log(k) + 1\end{aligned}$$

Ce qu'il fallait démontrer.
